

NutriFlow

Plataforma SaaS de gestión nutricional

Acta de entrega N° 2

Evaluación Parcial 2 (EV2)

Asignatura: Taller Aplicado de Programación

Sección: 001D

Docente: Jose Ignacio Campos Arévalo

Integrante 1: Benjamín González

Integrante 2: Álvaro Uribe

Fecha de entrega: [Indicar fecha de entrega]

1. Descripción general de la evaluación

La presente Evaluación N° 2 corresponde al segundo hito formal de entrega del proyecto académico NutriFlow, desarrollado en el marco de la asignatura Taller Aplicado de Programación. En esta instancia, el equipo compuesto por Benjamín González y Álvaro Uribe transitó desde la fase de ideación y diseño conceptual, establecida en el Acta de Entrega N° 1, hacia la implementación técnica del producto: arquitectura de software, configuración de entornos de desarrollo y pruebas, y el inicio de la codificación funcional del sistema.

La Evaluación Parcial 2 (EV2) constituye el 35% de la ponderación de la nota final de la asignatura. El encargo se desarrolló durante cinco semanas de trabajo continuo, culminando con la entrega de los repositorios y el informe escrito, seguida de una presentación de avance y defensa oral ante el docente guía, con un bloque de 40 minutos dividido en 30 minutos de exposición técnica y 10 minutos de preguntas y respuestas.

El proyecto continúa siendo desarrollado por un equipo de dos integrantes que asumen roles de Fullstack Developer, situación que exigió una distribución optimizada de las cargas de trabajo de arquitectura, backend, frontend y base de datos respecto al estándar habitual de tres integrantes.

1.1 Resultados de aprendizaje e indicadores de logro

El Resultado de Aprendizaje evaluado (RA2) exige que el equipo desarrolle una solución innovadora, fundamentada en componentes de software escalables, que vincule los intereses y necesidades del cliente final, resguardando los aspectos éticos de calidad, privacidad de los datos clínicos y seguridad de la solución. Este resultado se desglosa en tres indicadores de logro:

- **IL 2.1:** Elaboración de documentos técnicos avanzados para diseñar la solución, empleando técnicas y herramientas de ingeniería de software que definan las características, atributos de calidad y funcionalidades del producto final.
- **IL 2.2:** Generación y estandarización de un ambiente de pruebas aislado, capaz de replicar con exactitud el comportamiento y la configuración del ambiente de producción en la nube.
- **IL 2.3:** Desarrollo tangible de la solución utilizando un ecosistema de tecnologías adecuado al problema planteado, garantizando un producto funcional, de alta disponibilidad, calidad y seguridad.

2. Diseño de la solución

Esta etapa se enfocó en consolidar la documentación de arquitectura y la capa de diseño estructural del sistema, asegurando que el desarrollo posterior no presente deuda técnica ni desalineamiento con los requisitos clínicos de los nutricionistas.

2.1 Arquitectura del sistema: Modelo de Vistas 4+1

Para documentar la arquitectura de NutriFlow se utilizó el Modelo de Vistas 4+1 de Philippe Kruchten, el cual permite observar el sistema desde múltiples perspectivas complementarias. La arquitectura general se fundamenta en una API híbrida bajo paradigma Cliente-Servidor, distribuyendo las cargas operativas en nodos de cómputo especializados en lugar de una aplicación monolítica.

Vista arquitectónica	Descripción y abordaje en NutriFlow	Componentes y tecnologías
Vista lógica (diseño)	Estructura estática del código en capas de responsabilidad funcional, aislando el acceso a datos de la lógica de negocio.	React.js (Vite), NestJS/TypeScript, FastAPI/Python.
Vista de procesos	Ejecución dinámica y concurrencia de la API híbrida mediante peticiones asíncronas REST.	HTTP asíncrono, validación JWT, caché Redis.
Vista de escenarios	Define los casos de uso desde la lógica clínica, mostrando la interacción entre capas para generar cálculos y pautas.	Ingreso de datos → NestJS → FastAPI → PostgreSQL.
Vista de desarrollo	Organización modular del código fuente en el sistema de control de versiones.	Repositorios separados por dominio: frontend, backend-core, backend-math.
Vista de despliegue	Distribución de componentes en nodos de hardware o instancias cloud, garantizando alta disponibilidad.	Contenedores Docker, base de datos en Supabase.

La decisión de bifurcar el backend en dos motores independientes (Backend-Core y Backend-Math) obedece a razones de rendimiento. NestJS, basado en un Event Loop de un solo hilo, es idóneo para orquestar la seguridad y miles de conexiones HTTP concurrentes, pero presenta vulnerabilidad ante operaciones intensivas de CPU. El cálculo simultáneo de fórmulas termodinámicas y el algoritmo de sugerencia de preparaciones bloquearían dicho Event Loop, por lo que la carga matemática se delegó a FastAPI.

2.2 Diseño visual y reglas de negocio clínico

El diseño contempló el desarrollo de wireframes y mockups interactivos para la Pizarra Visual Drag & Drop, que reemplaza las hojas de cálculo tradicionales por una interfaz donde el nutricionista distribuye porciones de alimentos en los tiempos de comida del paciente.

La validación del prototipo con profesionales de la nutrición clínica derivó en nuevas reglas de negocio, entre ellas un mecanismo de auto-completado (“auto-fill”) que retiene los datos antropométricos del paciente entre módulos, con control visual para activar o desactivar dicha retención. Asimismo, se amplió la taxonomía de grupos de alimentos a 13 categorías, incorporando distinciones como verduras de libre consumo, carnes magras y altas en grasa, y lácteos diferenciados por contenido graso.

Se incorporó también el porcentaje de adecuación calórica, que permite al profesional definir rangos de tolerancia clínica (por ejemplo, 90%–110% respecto a la meta calórica) para declarar una pauta como válida sin forzar ajustes matemáticos irreales.

3. Entornos de desarrollo, pruebas e infraestructura

El Indicador de Logro 2.2 evalúa la capacidad del equipo para generar un ambiente de pruebas que replique de forma fiel y aislada las características del ambiente de producción. Para ello se diseñó un protocolo de configuración de infraestructura y sistemas de respaldo.

3.1 Infraestructura local y gestión de paquetes

El entorno de desarrollo se optimizó para arquitectura Apple Silicon, gestionando el aprovisionamiento base mediante Homebrew. Se estandarizó el uso de Python 3.13 para el backend matemático, evitando versiones experimentales que introducen riesgos de incompatibilidad en librerías científicas. La gestión de entornos virtuales de Python se realiza con uv, instalador rápido de Astral, mientras que PostgreSQL 15 y Redis se ejecutan como servicios nativos administrados mediante interfaces como DBeaver o TablePlus.

3.2 Contenerización y paridad de entornos

NutriFlow implementa contenedores Docker para garantizar paridad entre desarrollo, pruebas y producción. El Dockerfile del motor matemático se basa en la imagen python:3.11-slim y expone el servicio Uvicorn en el puerto 8000. Las variables de entorno sensibles se gestionan mediante archivos .env diferenciados por ambiente (.env.test, .env.prod), excluidos del repositorio mediante .gitignore.

3.3 Plan de copias de seguridad y enmascaramiento de datos

Se diseñó un Plan Integral de Copias de Seguridad para resguardar la información de producción alojada en Supabase, complementando los respaldos automáticos del proveedor con respaldos externos auditables. El ciclo de sincronización consta de tres etapas:

- **Extracción:** uso de pg_dump sobre la conexión de producción para generar un volcado binario comprimido (formato .backup).
- **Saneamiento:** anonimización de datos sensibles (Data Masking) y limpieza del esquema local antes de la importación, en cumplimiento de los principios éticos de protección de datos clínicos.
- **Restauración:** inyección del volcado saneado en el ambiente local o de pruebas mediante pg_restore.

Este proceso fue automatizado mediante un flujo de GitHub Actions que ejecuta `pg_dump` de forma programada o bajo demanda, conservando los respaldos como artefactos encriptados con una política de retención de 7 días.

4. Desarrollo de la aplicación y lógica de backend

Bajo el Indicador de Logro 2.3, se desarrolló de forma progresiva el núcleo funcional de NutriFlow, traduciendo los requerimientos conceptuales en servicios interactivos y operacionales.

4.1 Refactorización del esquema relacional

Una revisión técnica del modelo de datos determinó separar la base de cálculos energéticos de la pauta visual, dado que un paciente posee una única Planificación pero puede requerir múltiples Pautas asociadas. Se modeló esta relación Uno a Muchos (1:N) en el ORM Prisma, utilizando identificadores UUID como llave primaria —contramedida contra ataques de enumeración— y borrado en cascada para mantener la integridad referencial. Este rediseño se reflejó en el frontend mediante un proceso de guardado en dos pasos: primero el registro de la planificación, y posteriormente el de la pauta asociada.

4.2 Flujo de comunicación Frontend–Core–Math

Se consolidó la comunicación entre los tres servicios de la arquitectura: el cliente React (puerto 5173), el Backend-Core en NestJS (puerto 3000) y el Motor Matemático en FastAPI (puerto 8000). NestJS actúa como intermediario, recibiendo los datos del paciente, delegando el cálculo a FastAPI y formateando la respuesta para su consumo en React, manteniendo las fórmulas clínicas encapsuladas y protegidas de exposición directa.

4.3 Motor de cálculos metabólicos en FastAPI

La capa matemática implementa validación estricta de datos mediante Pydantic, asegurando que los parámetros antropométricos sean numéricos y fisiológicamente coherentes antes de procesar el cálculo. El servicio de cálculo incorpora siete fórmulas clínicas (Harris-Benedict, Mifflin-St Jeor, Katch-McArdle, Cunningham, Owen, Oxford y FAO/OMS), consolidando un promedio matemático de la Tasa Metabólica Basal. Se iniciaron además las bases del módulo Cuadrador Dinámico de Macronutrientes, validando que la sumatoria de porcentajes de macronutrientes sea exactamente igual al 100%.

4.4 Autenticación, seguridad y privacidad

Se implementó un sistema de autenticación basado en JSON Web Tokens (JWT) mediante la librería `@nestjs/jwt`, optando por una solución manual en lugar de las herramientas nativas de Supabase Auth. Esta arquitectura stateless permite validar la identidad del usuario sin consultas repetitivas a la base de datos. El módulo de autenticación contempla registro con encriptación de contraseñas, confirmación de correo electrónico, inicio de sesión y recuperación de acceso.

5. Plan de pruebas de software y aseguramiento de calidad

En cumplimiento del criterio de inclusión de pruebas operativas, de validación y verificación, el desarrollo del código se acompañó de un plan de pruebas estructurado en dos niveles.

5.1 Verificación operativa y testing de APIs

Aprovechando la documentación automática OpenAPI de FastAPI, se utilizó Swagger UI (<http://127.0.0.1:8000/docs>) para ejecutar pruebas operacionales sobre el endpoint de cálculo de

TMB, certificando que las siete fórmulas clínicas se procesan correctamente y devuelven resultados consistentes. Las pruebas de integración entre frontend, NestJS y FastAPI se realizaron con Postman, Thunder Client y la herramienta ProxyMan, verificando la correcta transmisión de tokens JWT a lo largo del flujo de datos.

5.2 Validación clínica integral

Las lógicas del Dashboard de Macronutrientes fueron cotejadas con los procedimientos y cálculos utilizados históricamente por el nutricionista patrocinador del proyecto, confirmando que los resultados y porcentajes de adecuación calórica satisfacen los esquemas de trabajo clínico real.

6. Síntesis de la evaluación N° 2

Este avance evidencia la transición exitosa del proyecto desde la fase de diseño conceptual (Acta N° 1) hacia la implementación técnica funcional. Se completó la documentación de arquitectura bajo el Modelo de Vistas 4+1 (IL 2.1), se configuraron ambientes de desarrollo y pruebas contenerizados con Docker y respaldos automatizados (IL 2.2), y se codificó un ecosistema funcional compuesto por React.js, NestJS, FastAPI, PostgreSQL y Prisma, con autenticación JWT y validación de datos mediante Pydantic (IL 2.3).

La asignación de tareas se mantiene equitativa entre ambos integrantes: Benjamín González lidera la implementación visual del frontend, mientras que Álvaro Uribe lidera la arquitectura de base de datos y el backend. La robustez técnica alcanzada y las pruebas de integración realizadas certifican el avance del proyecto hacia su etapa de cierre.

7. Firmas

Los abajo firmantes certifican que los entregables descritos en el presente documento fueron elaborados conforme a los estándares exigidos por la asignatura y representan fielmente el trabajo realizado durante la Evaluación N° 2 del proyecto NutriFlow.

Benjamín González
Integrante – Frontend

Álvaro Uribe
Integrante – DB & Backend